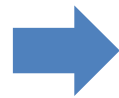


学习内容

CONTENTS

- 
- 1 机器学习概述
 - 2 线性回归与逻辑回归
 - 3 支持向量机
 - 4 聚类与降维
 - 5 决策树
 - 6 强化学习

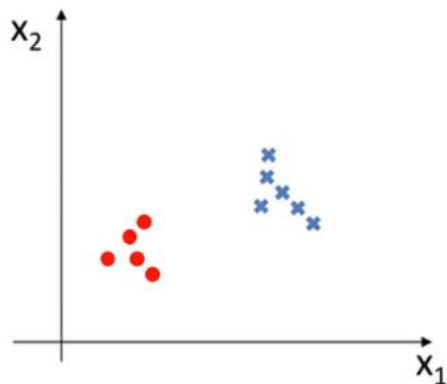


有监督学习

利用带有标签的数据进行模型训练

用学到的规律来预测新数据

“五年高考三年模拟”



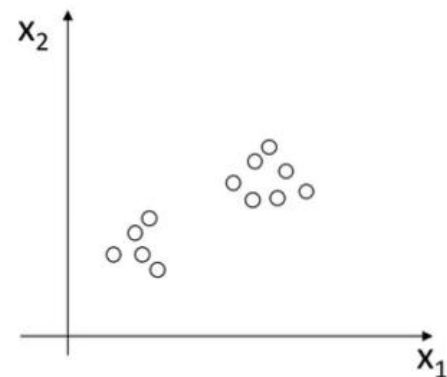
无监督学习

对没有标签的数据进行自主探索

发现内在规律和结构

用总结出的规律来预测新数据

“读书百遍其义自见”



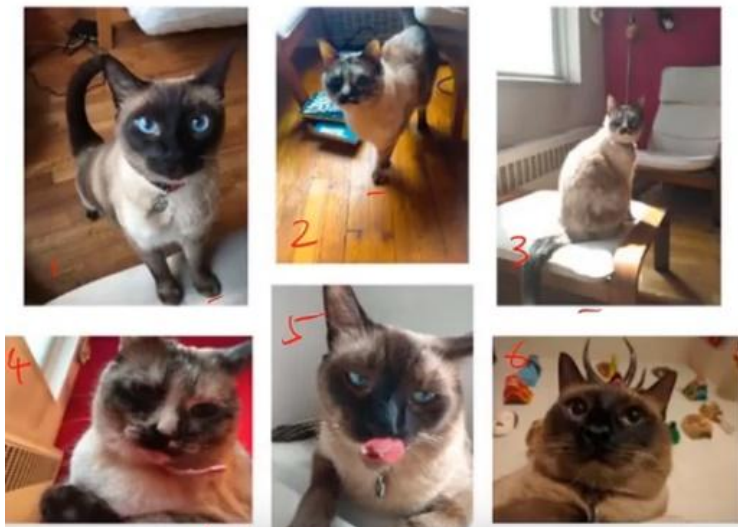
没有对与错（没有标签），寻找数据的共同点

优点：

- 无需标签，数据获取成本低
大幅度扩大数据样本
- 不受监督/偏见的约束
能发现未知模式
- 可作预处理工具
降维、特征提取

缺点：

目标模糊、评估困难.....



根据自己喜欢的方式将上图六个实例分为**两组**
没有对错之分，那么计算机该如何去分呢？

方式一：站着（1,2）、坐着（3,4,5,6）

方式二：全身（1,2,3）、半身（4,5,6）

方式三：蓝眼球（1,2,3,5）、不是蓝眼球（4,6）

聚类分析

相似数据归类

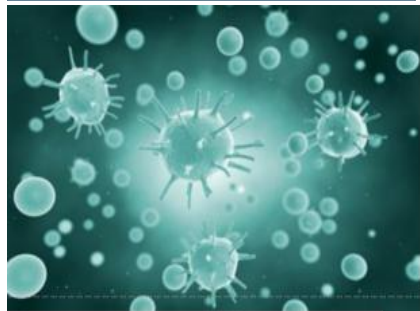
比如：用户分类（相似消费习惯）



维度缩减

精简数据，保留主要信息

比如：数据可视化



关联规则

挖掘数据中的隐藏关联

比如：搭配销售（泡面+火腿肠）



聚类 (Clustering) 的思想——物以类聚，人以群分



核心任务：无标签分群

面对无标签数据，基于数据分布特征，自动将相似数据归并到同一个簇内，实现数据的自然分组。



直观比喻：自然抱团

就像人与人之间会因为兴趣相投而形成圈子，聚类算法让“相似的东西”自动归成一类，形成紧密的群体。



生活例子：衣物分类

整理衣柜时，我们会根据功能和属性，将衣物自然地分为上衣、裤子、外套等类别，这就是聚类的日常体现。

聚类 (Clustering) 的思想——物以类聚，人以群分

上衣区
(T恤、衬衫)

T恤

衬衫

裤子区
(牛仔裤、休闲裤)

牛仔裤

休闲裤

外套区
(夹克、风衣)

夹克

风衣

聚类问题的本质与目标



核心定义：数据的自动分组

按照**特定标准**将数据集分割成不同的类或簇，本质是无监督的类别归纳过程。



目标一：簇内相似性最大化

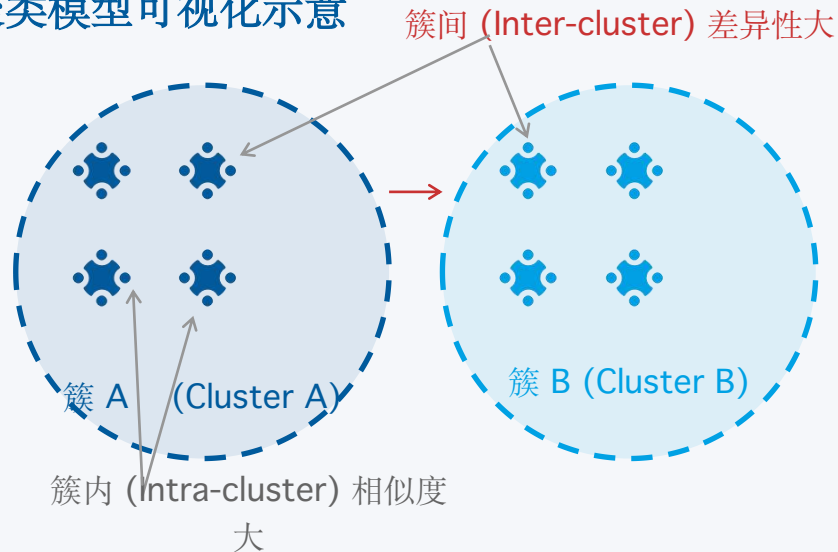
同一簇内的数据对象特征高度一致，如同“一家人”特征相似。



目标二：簇间差异性最大化

不同簇之间的数据对象特征显著不同，如同“不同家庭”特征各异。

聚类模型可视化示意



输出特点：离散类别标签，用于预测新样本归属

- 聚类算法可以应用于探索性数据挖掘、统计分析、生物信息学、数据压缩、计算机图像识别、医学影像分析等，在商业领域可以用来做市场研究、商品归类，在社会科学领域可以用来做犯罪区域分析等等。
 - 可以作为一个单独过程，寻找数据内在的分布结构
 - 可以作为分类、稀疏表示等其它学习任务的前驱过程



数据挖掘与统计

揭示数据内在结构，发现隐藏模式与趋势。



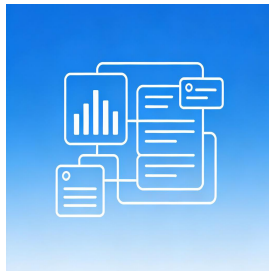
生物信息学

基因表达分析、蛋白质结构预测及疾病分类。



图像识别与医学影像

图像分割、目标检测及病灶识别的基础技术。



商业市场研究

客户分群、市场细分、商品推荐与竞品分析。



社会科学分析

识别犯罪热点区域，辅助资源调配与政策制定。

图像分割：将一幅图像分割成不同区域，理想情况下每个区域对应图像中一个物体。

图像中每一个像素点是一个3维向量，对应 $[R, G, B]$ 像素值。给定聚类中类别个数 K ，算法用 K 个不同的颜色来表示原来的图像，每个像素点用 K 个颜色中一个表示。

原图

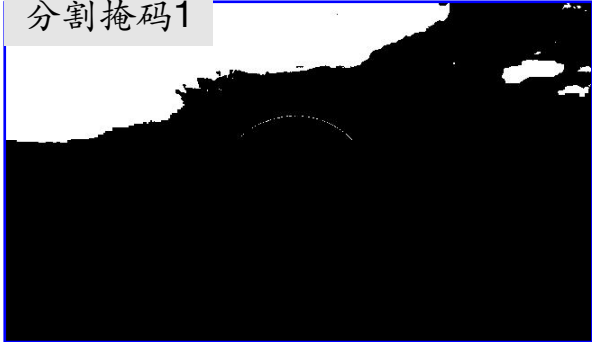


K=4分割

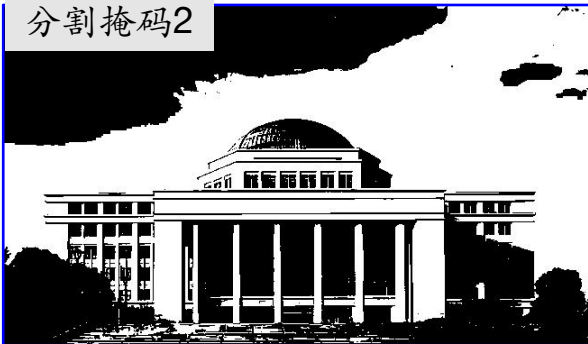


图

分割掩码1



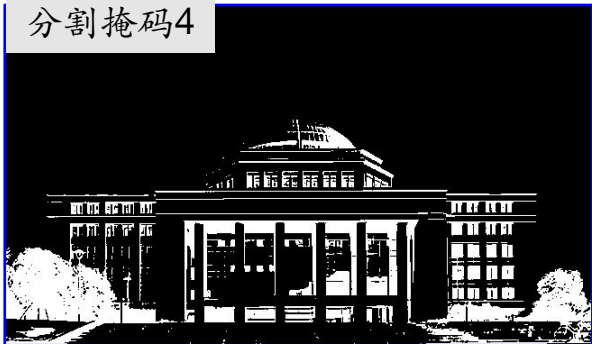
分割掩码2



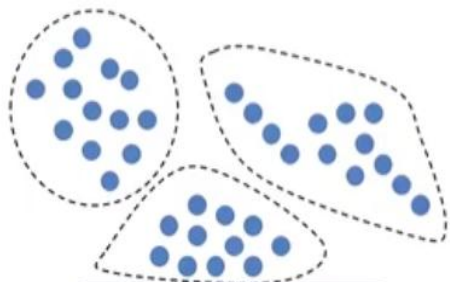
分割掩码3



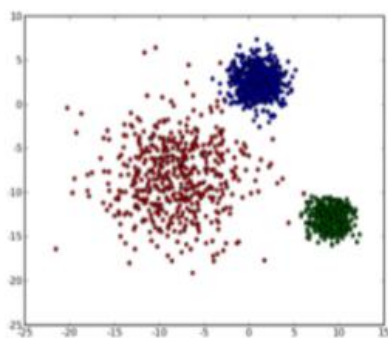
分割掩码4



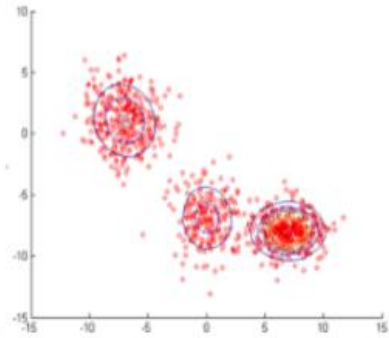
- 典型的聚类算法:
 - **K-Means算法**: 简单、易于实现, 对于大规模数据集, 相对高效; 当簇是球形且大小相近时, 聚类效果好
 - **高斯混合聚类**: 使用概率模型, 可以处理非球形簇, 但计算更复杂
 - **密度聚类 (如Mean-Shift, DB-SCAN)**: **DB-SCAN**不需要指定簇数, 可以处理任意形状的簇, 对噪声不敏感。



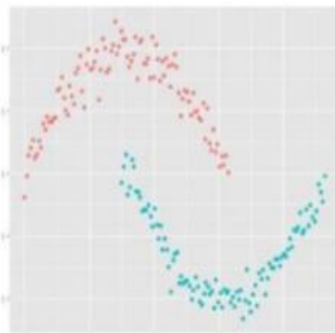
无监督学习



K-Means算法



高斯混合聚类



DB-SCAN算法

K-means（又称**k-均值**或**k-平均**）聚类算法目的：把 n 个数据点按照分布分成 K 类（ K 是人为提前设定的），即希望通过聚类算法得到 **K 个中心点**，以及**每个数据点属于哪个中心点的划分**。

- **中心点**可以通过迭代算法来找到，满足条件：所有的数据点到聚类中心的距离之和是最小的。
- 中心点确定后，每个数据点属于离它最近的中心点。

Q1：数据点到中心点的距离如何计算？

- 我们一般选择欧式距离，也可以其他距离度量，例如曼哈顿距离。

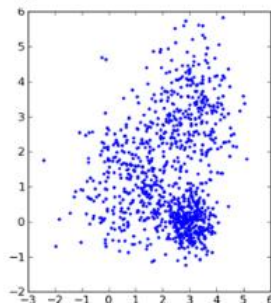
Q2：中心点是否唯一，或者说，是不是存在全局最优解？

- 对于多个中心点的情况，全局最优是一个相当难的问题。理论上存在一个全局最优解，但是不一定能找到。既然全局最优解不好找，那我们退而求其次，看能不能找到局部最优解。

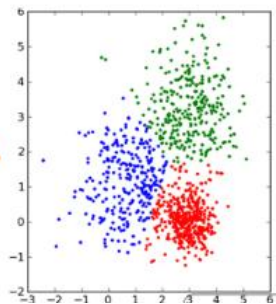
Q3：聚类结果如何表示？

- 采用空间分割的方式：将空间分割成多个多边形，每个多边形对应一个 cluster 中心。

Original Unclustered Data



Clustered Data





- ◆ 假设共有 N 个数据： $\mathbf{x}_1, \dots, \mathbf{x}_N$ ，我们想把它们分成 k 个簇。
- ◆ 记 k 个簇 $C_1, C_2, \dots, C_k$ ，它们的中心为 $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_k$ 。每个簇的样本数目为 $N_1, N_2, \dots, N_k$ 。则使用**欧式距离平方和误差**作有如下的目标函数：

$$J(\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_k) = \frac{1}{2} \sum_{j=1}^K \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$$

$$J(\mu_1, \mu_2, \dots, \mu_k) = \frac{1}{2} \sum_{j=1}^K \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \mu_j\|_2^2$$

◆ 固定簇中心，优化归属关系。就数据 $\mathbf{x}_i$ 而言，其所属的簇应满足（一个数据仅属于一个簇）：

$$\operatorname{argmin}_j \|\mathbf{x}_i - \mu_j\|_2^2$$

◆ 固定从属关系，优化簇中心。就簇中心而言，对关于从 $\mu_1, \mu_2, \dots, \mu_k$ 的函数求偏导，这里的求偏导

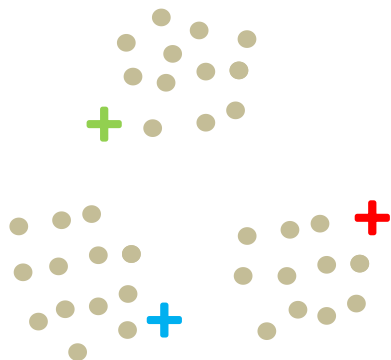
是对第 j 个簇中心 μ_j 求的偏导。故而其驻点为：

$$\begin{aligned} \frac{\partial J}{\partial \mu_j} &= - \sum_{\mathbf{x}_i \in C_j} (\mathbf{x}_i - \mu_j) \stackrel{\Delta}{=} 0 \\ \Rightarrow \mu_j &= \frac{1}{N_j} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i \end{aligned}$$

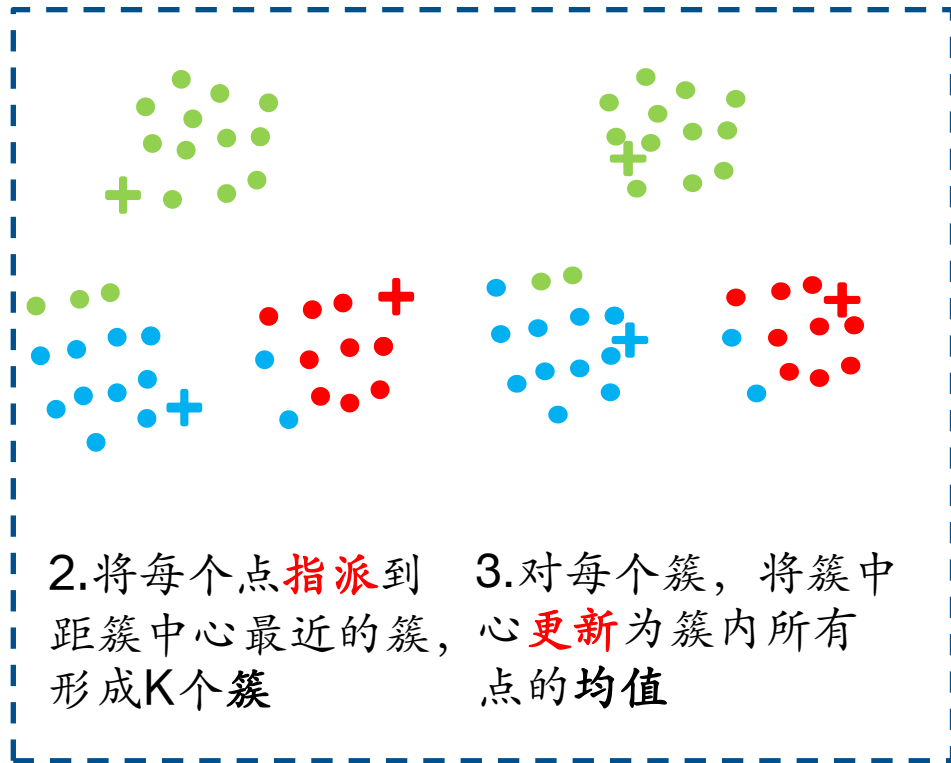
推导：

$$\begin{aligned} \frac{\partial J}{\partial \mu_j} &= \frac{\partial \frac{1}{2} \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \mu_j\|_2^2}{\partial \mu_j} \\ &= \frac{\partial \frac{1}{2} \sum_{\mathbf{x}_i \in C_j} (\mathbf{x}_i - \mu_j)^2}{\partial \mu_j} \\ &= \sum_{\mathbf{x}_i \in C_j} (\mathbf{x}_i - \mu_j) \cdot (-1) \\ &= - \sum_{\mathbf{x}_i \in C_j} (\mathbf{x}_i - \mu_j) \end{aligned}$$

4. **重复**2-3步，直至簇中心不再变化

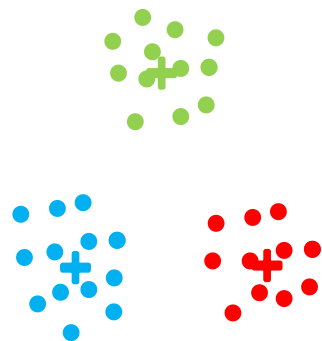


1. **选择**K个点作为
初始化簇中心



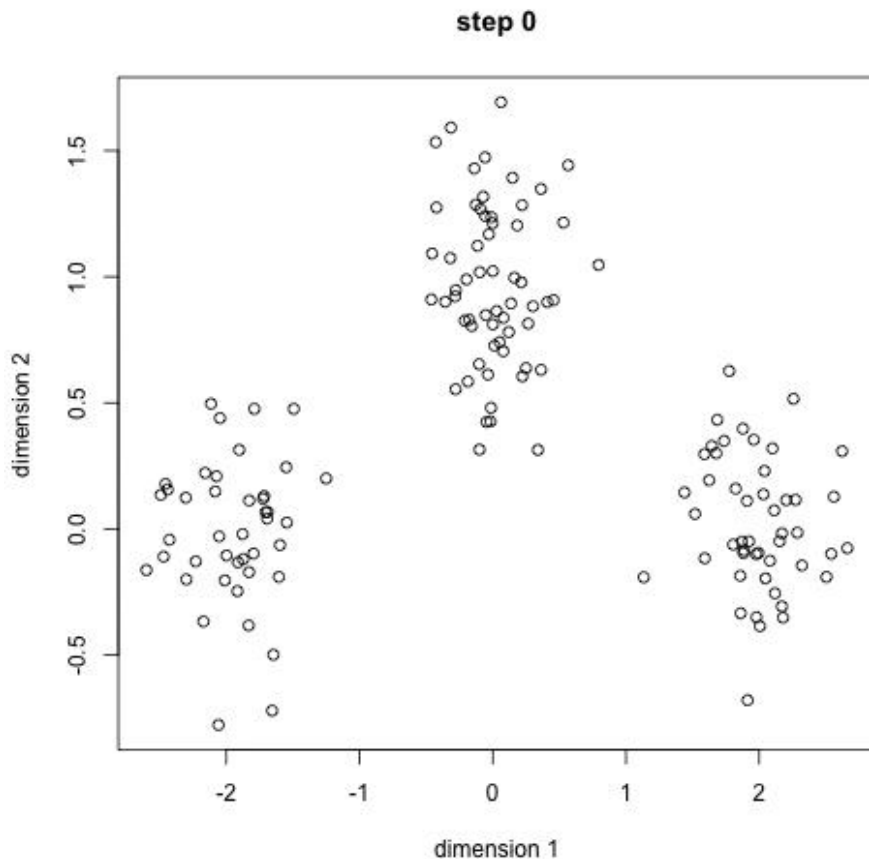
2. 将每个点**指派**到
距簇中心最近的簇，
形成K个簇

3. 对每个簇，将簇中
心**更新**为簇内所有
点的均值



收敛

- ◆ 因为，此算法簇心就是簇内点的均值，因此唤作K-均值(K-means)算法或K-质心算法。它是一种无监督学习方法，是最普及的聚类算法。



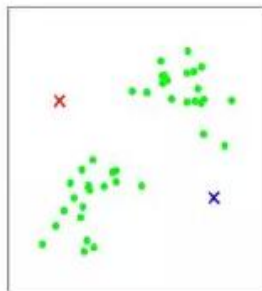
图(c)和图(e)中哪个图的簇内平方和(即各数据点到其所在簇中心的欧式距离平方和)小?

A 图(c)

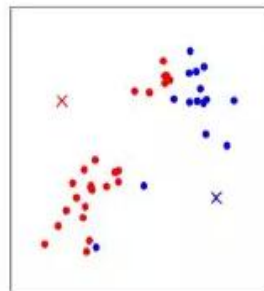
B 图(e)



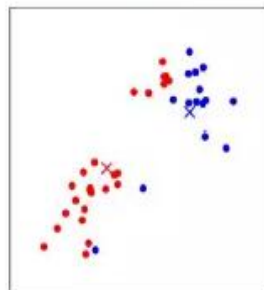
(a)



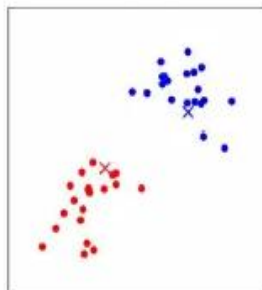
(b)



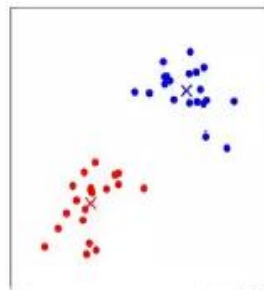
(c)



(d)



(e)



(f)

提交

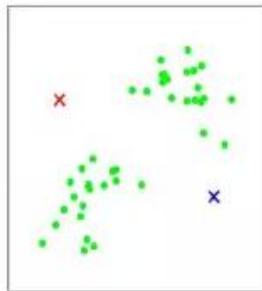
图(c)和图(e)中哪个图的簇内平方和(即各数据点到其所在簇中心的欧式距离平方和)小?

☐ A 图(c)

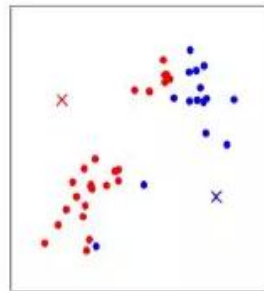
☒ B 图(e)



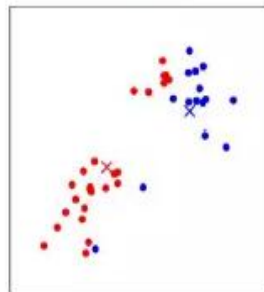
(a)



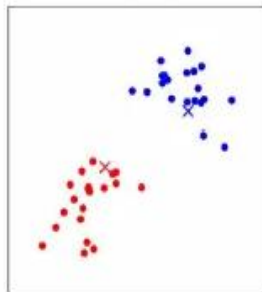
(b)



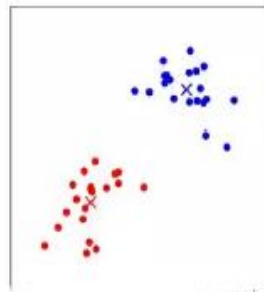
(c)



(d)



(e)



(f)

提交

◆ 用K-means将二维点 $p_1 = (1,2)$, $p_2 = (1,5)$, $p_3 = (2,4)$, $p_4 = (4,3)$, $p_5 =$

$(5,2)$, $p_6 = (6,1)$ 聚成两个簇。如果, 首先选择 p_1 , p_2 为簇1、簇2的中心, 那

✎ 1)分配所有二维点, 属于簇1的点有_____, 其余点属于簇2;

➤ 2)更新簇1、簇2的中心分别为_____、_____;

➤ 3)分配所有二维点, 属于簇1的点有_____, 其余点属于簇2;

➤ 4)更新簇1、簇2的中心分别为_____、_____。

提交

- ◆ 用K-means将二维点 $p_1 = (1,2)$, $p_2 = (1,5)$, $p_3 = (2,4)$, $p_4 = (4,3)$, $p_5 = (5,2)$, $p_6 = (6,1)$ 聚成两个簇。如果, 首先选择 p_1 , p_2 为簇1、簇2的中心, 那么
- 1) 分配所有二维点, 属于簇1的点有 p_1 、 p_4 、 p_5 、 p_6 , 其余点属于簇2;
 - 2) 更新簇1、簇2的中心分别为 $(4,2)$ 、 $(3/2, 9/2)$;
 - 3) 分配所有二维点, 属于簇1的点有 p_4 、 p_5 、 p_6 , 其余点属于簇2;
 - 4) 更新簇1、簇2的中心分别为 $(5,2)$ 、 $(4/3, 11/3)$ 。

- K-Means 算法实例

接下来以下表的西瓜数据集4.0为例，来演示k-均值算法的学习过程。
将编号为 i 的样本称为 x_i 。

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

假定聚类簇数 $k=3$

- k -均值算法实例

$$\|x_1 - \mu_1\|_2 = \sqrt{(0.697 - 0.403)^2 + (0.460 - 0.237)^2} = 0.369$$

假定聚类簇数 $k=3$ ，算法开始时，随机选择3个样本 x_6, x_{12}, x_{27} 作为初始均值向量，即

$$d_{ji} = \|\mathbf{x}_j - \boldsymbol{\mu}_i\|_2$$

$$\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.533; 0.472)$$

考察样本 $x_1 = (0.697; 0.460)$ 它与当前均值向量 μ_1, μ_2, μ_3 的距离分别为 0.369, 0.506, 0.166，因此 x_1 将被划入簇 C_3 中。类似的，对数据集中的所有样本考察一遍后，可得当前簇划分为

$$C_1 = \{x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{15}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$

$$C_3 = \{x_1, x_2, x_3, x_4, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

于是，可以从分别求得新的均值向量

$$\mu'_i = \frac{\sum_{j=1}^m \gamma_{ij} x_j}{\sum_{j=1}^m \gamma_{ij}} = \frac{\sum_{x_j \in C_i} x_j}{|C_i|}$$

$$\mu'_1 = (0.473; 0.214), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.623; 0.388)$$

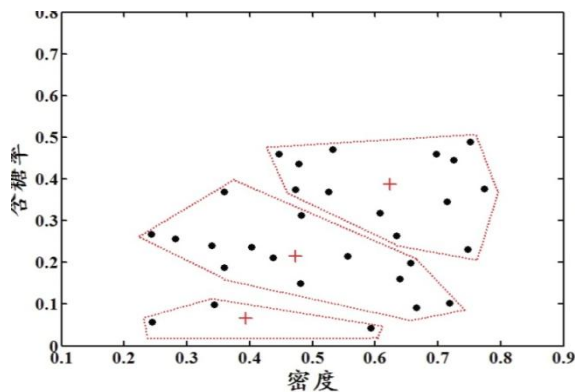
不断重复上述过程，如下图所示。

- 聚类结果:

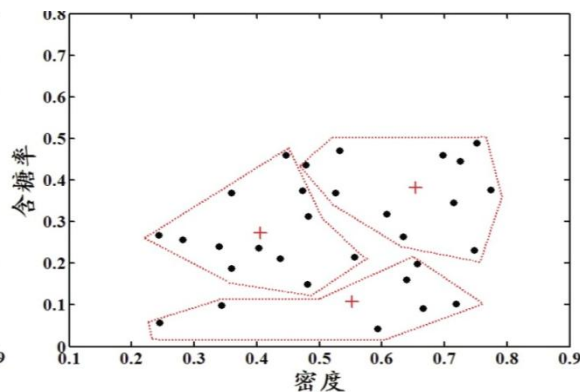
$$\min E = \sum_{i=1}^k \sum_{j=1}^m V_{ji} \|x_j - \mu_i\|_2^2$$

$$V_{ji} = \begin{cases} 1 & x_j \in C_i \\ 0 & x_j \notin C_i \end{cases}$$

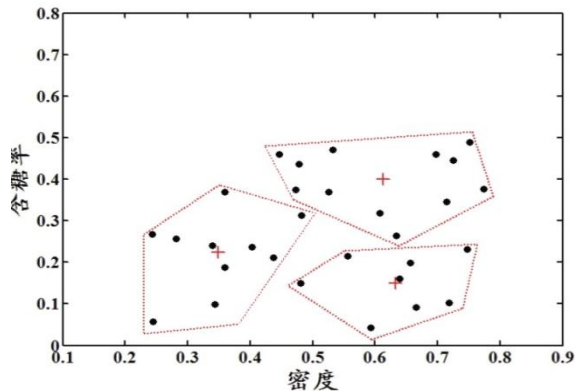
特点：采用距离作为相似性的评价指标(非凸函数，易收敛到局部最小)，同一个簇内的对象相似度较高；而不同簇内的对象相似度较小。



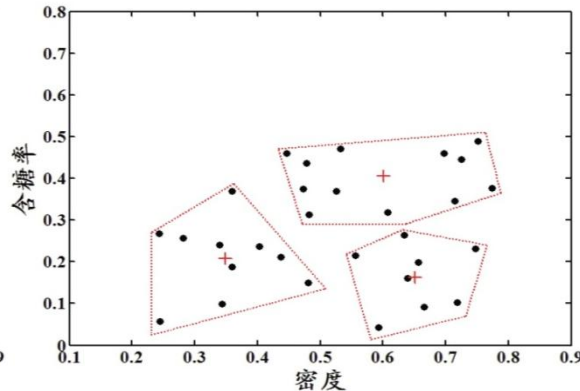
(a) 第一轮迭代后



(b) 第二轮迭代后



(c) 第三轮迭代后



(d) 第四轮迭代后

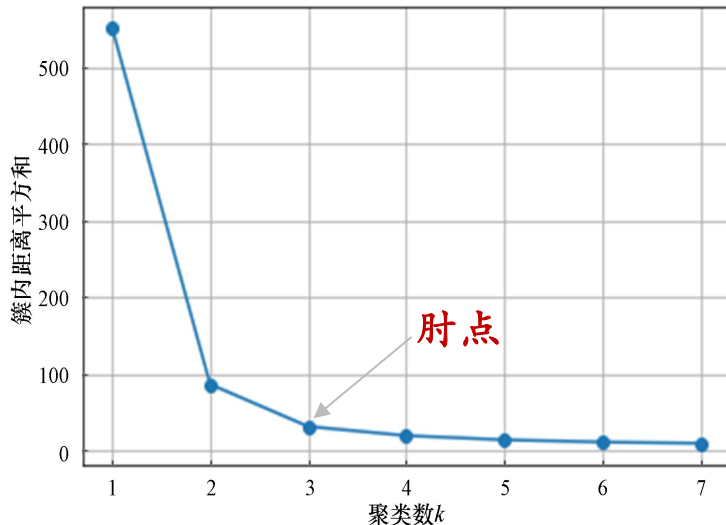
K-Means的主要**优点**：

- ✓ 原理比较简单，实现也是很容易，收敛速度快
- ✓ 聚类效果较优
- ✓ 算法的可解释度比较强
- ✓ 主要需要调参的参数仅仅是簇数 k

K-Means的主要**缺点**：

- ✓ K值的选取不好把握
- ✓ 不平衡数据集的聚类效果不佳
- ✓ 采用迭代方法，得到的结果只是局部最优。
- ✓ 对噪音和异常点比较敏感。

聚类数 k 和簇内距离平方和的关系图



$k=3$ 时，簇内距离平方和的下降率突然变缓，
可以考虑选择 $k=3$ 作为聚类数量。

1) K怎么选取: **肘部法**

2) 初始聚类中心怎么选:

K-means++ (选择彼此距离最远的点)

KMeans++ 算法(2007年, D. Arthur等): 初始聚类中心的选择方法, “距离越远的点被选中的概率越高”

具体步骤:

1、从样本集中随机选择一个样本 c_1 作为第1个初始中心;

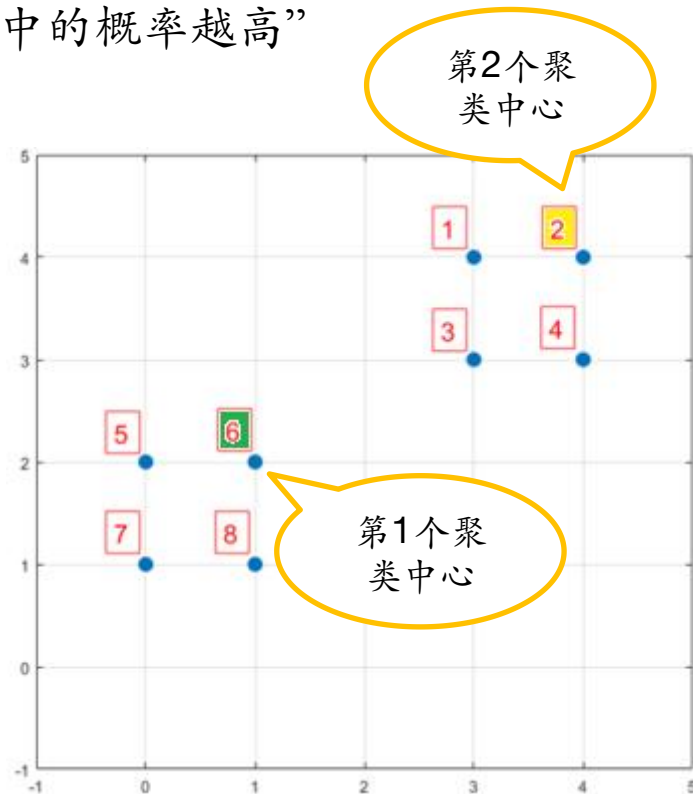
2、计算各样本 x 到各聚类中心的最近距离 $d(x)$

3、以概率 $\frac{d(x)^2}{\sum_{x \in X} d(x)^2}$ 选择一个新样本 c_i 加入聚类

中心集合中, 其中距离值 $d(x)$ 越大, 被选中的可能性越高; ;

4、重复步骤2和3直到选满 K 个初始中心;

5、后续步骤执行与经典 KMeans 相同的迭代过程



#1. 导入必要的库

```
from sklearn.cluster import KMeans
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
```

#2. 准备数据

```
iris = datasets.load_iris() #加载iris数据集
X = iris.data               #特征数据
y = iris.target             #目标数据
```

#3. 数据标准化

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

#4. 构建及训练聚类模型

```
kmeans = KMeans(n_clusters=3, init='k-means++')
kmeans.fit(X_scaled) # kmeans聚类
```

#5. 输出聚类结果

```
print('聚类中心: \n', kmeans.cluster_centers_)
print('聚类标签: \n', kmeans.labels_)
print('簇内平方和: ', kmeans.inertia_)
```



山鸢尾



变色鸢尾



维吉尼亚鸢尾

花萼长度	花萼宽度	花瓣长度	花瓣宽度	品种 (标签)
5.1	3.3	1.7	0.5	0 (山鸢尾)
5.0	2.3	3.3	1.0	1 (变色鸢尾)
6.4	2.8	5.6	2.2	2 (维吉尼亚鸢尾)

KMeans 类的主要参数有:

- 1) **n_clusters**: **k值**, 一般需要多试一些值以获得较好的聚类效果。
- 2) **max_iter**: 最大的迭代次数, 默认是300。
- 3) **n_init**: 用不同的初始化聚类中心运行算法的次数, 默认是10。
- 4) **init**: 初始聚类中心选择的方式, 有'random'、'**k-means++** (默认)'等
- 5) **algorithm**: 有“auto” (默认)、“full” or “elkan” (数据稠密) 三种选择。

6. 可视化聚类结果

```

y_pred = kmeans.predict(X_scaled)
centers = kmeans.cluster_centers_
plt.rcParams['axes.unicode_minus'] = False

plt.figure(figsize=(10,5))
plt.subplot(121)
...
plt.subplot(122)
for label in [0, 1, 2]:
    mask = (y_pred == label)
    plt.scatter(X_scaled[mask, 0],
                X_scaled[mask, 1], label=f'类别{label+1}')
plt.scatter(centers[:,0], centers[:,1], s=200,
            marker='*', c='red', label='Centroids')

plt.title('聚类结果')
plt.xlabel('花萼长度')
plt.ylabel('花萼宽度')
plt.legend()
plt.show()

```



山鸢尾

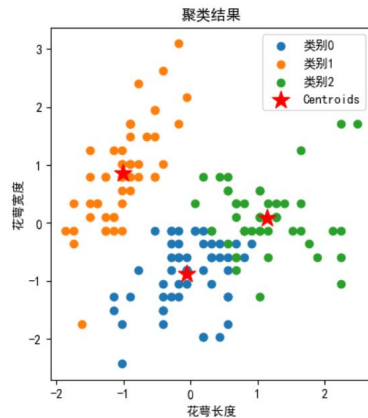
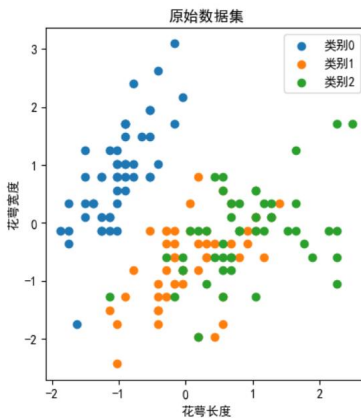


变色鸢尾



维吉尼亚鸢尾

花萼长度	花萼宽度	花瓣长度	花瓣宽度	品种 (标签)
5.1	3.3	1.7	0.5	0 (山鸢尾)
5.0	2.3	3.3	1.0	1 (变色鸢尾)
6.4	2.8	5.6	2.2	2 (维吉尼亚鸢尾)



k-均值对Iris数据集聚类的效果



山鸢尾



变色鸢尾



维吉尼亚鸢尾

花萼长度	花萼宽度	花瓣长度	花瓣宽度	品种 (标签)
5.1	3.3	1.7	0.5	0 (山鸢尾)
5.0	2.3	3.3	1.0	1 (变色鸢尾)
6.4	2.8	5.6	2.2	2 (维吉尼亚鸢尾)

#7. 矫正结果

```

y_corrected = []
for i in y_pred:
    if i == 0:
        y_corrected.append(1)
    elif i == 1:
        y_corrected.append(0)
    elif i == 2:
        y_corrected.append(2)

```

8. 聚类结果评估对比

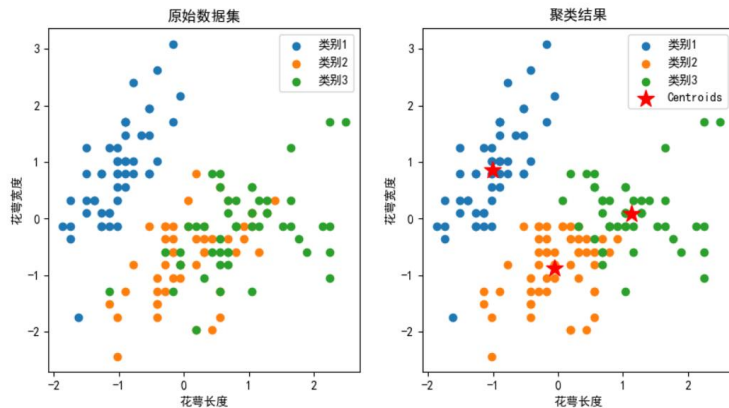
```

from sklearn.metrics import accuracy_score
print('调整前, 正确率: ', accuracy_score(y, y_pred))
print('调整后, 正确率: ', accuracy_score(y_corrected, y))

```

调整前, 正确率: 0.24

调整后, 正确率: 0.8333333333333334





山鸢尾



变色鸢尾



维吉尼亚鸢尾

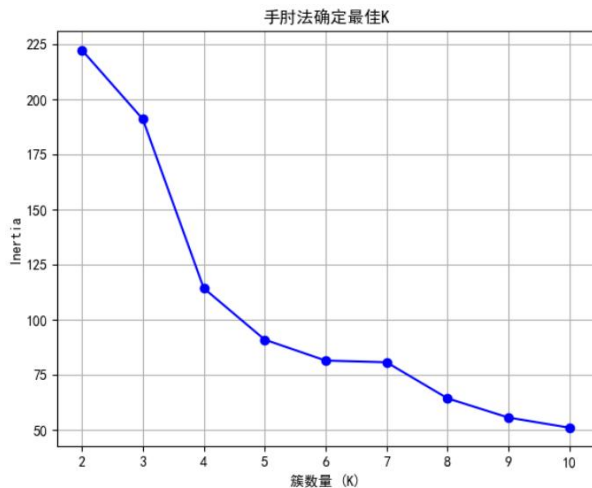
花萼长度	花萼宽度	花瓣长度	花瓣宽度	品种 (标签)
5.1	3.3	1.7	0.5	0 (山鸢尾)
5.0	2.3	3.3	1.0	1 (变色鸢尾)
6.4	2.8	5.6	2.2	2 (维吉尼亚鸢尾)

9. 绘制纠正后的聚类结果

...

#10. 使用手肘法确定最佳聚类数K

```
inertia = []  
K_range = range(2, 11)  
for k in K_range:  
    kmeans = KMeans(n_clusters=k, init='k-means++',  
                    random_state=42)  
    kmeans.fit(X_scaled)  
    inertia.append(kmeans.inertia_)
```



- 聚类分析的度量指标:

- **内部指标**: 不借助任何外部参考, 只用参与聚类的样本评判聚类结果好坏

- **外部指标**: 用事先指定的聚类模型作为参考来评判聚类结果的好坏

- **聚类的目标**: 得到较高的簇内相似度和较低的簇间相似度, 使得簇间的距离尽可能大, 簇内样本与簇中心的距离尽可能小

- 聚类得到的簇可以用聚类中心、簇大小、簇密度和簇描述等来表示

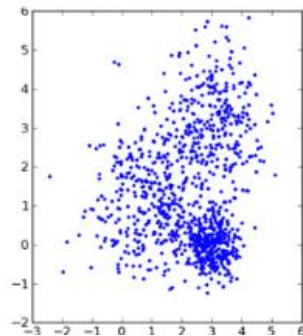
- **聚类中心**: 一个簇中所有样本点的均值(质心)

- **簇大小**: 簇中所含样本的数量

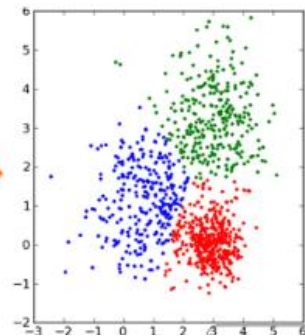
- **簇密度**: 簇中样本点的紧密程度

- **簇描述**: 簇中样本的业务特征

Original Unclustered Data



Clustered Data



- 轮廓系数:

对于样本 i , 1) 计算该样本到同簇其它样本的平均距离 $a(i)$, 2) 该样本到其距离最近的其它簇中所有样本的平均距离 $b(i)$, **样本 i 的轮廓系数**:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

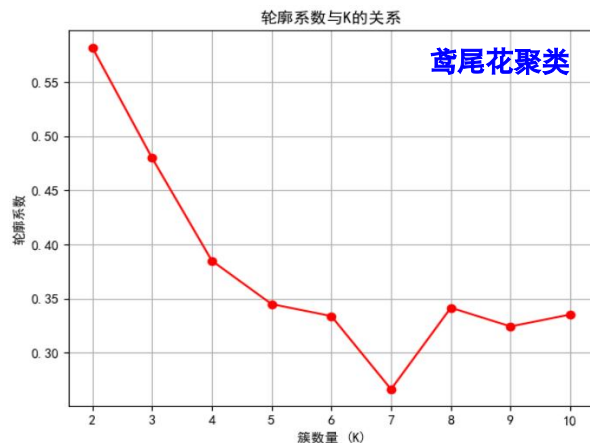
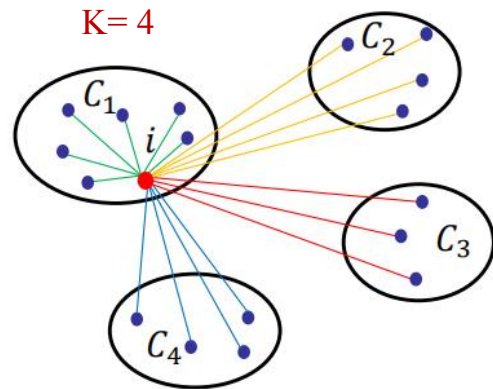
将所有样本的轮廓系数取算术平均值, 作为**聚类划分的性能指标**。

轮廓系数的区间为: $[-1, 1]$

-1: 聚类效果差, 1: 聚类效果好。

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, y_pred) #y_pred为聚类标签
print(f"\n轮廓系数 (Silhouette Score) : {score:.3f}")
```

轮廓系数 (Silhouette Score) : 0.460



- 对于含有 n 个样本点的数据集 S ，其中的两个不同样本点 (x_i, x_j) ，假设 C 是聚类算法给出的簇划分结果， P 是外部参考模型给出的簇划分结果。那么对于样本点 x_i, x_j 来说，存在以下四种关系：

SS: x_i, x_j 在 C 和 P 中属于相同的簇。

SD: x_i, x_j 在 C 中属于相同的簇，在 P 中属于不同的簇。

DS: x_i, x_j 在 C 中属于不同的簇，在 P 中属于相同的簇。

DD: x_i, x_j 在 C 和 P 中属于不同的簇。

a, b, c, d 分别表示 SS, SD, DS, DD 所对应的关系数目，由于 x_i, x_j 之间的关系必定存在于四种关系中的一种，且仅能存在一种关系

- Rand 统计量 (Rand Statistic)

$$R = \frac{a+d}{a+b+c+d}$$

/正确率 ACC

- F 值 (F-measure)

$$P = \frac{a}{a+b} \quad R = \frac{a}{a+c}$$

P 表示准确率， R 表示召回率。

$$F = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

β 是参数，当 $\beta = 1$ 时，就是最常见的 $F1 - measure$

$$\frac{2}{F1 - measure} = \frac{1}{P} + \frac{1}{R}$$

$$F1 - measure = \frac{2PR}{P + R}$$

```
from sklearn import metrics
from munkres import Munkres
R=metrics.accuracy_score(label, pred)
```

注意：使用 [匈牙利算法 Kuhn-Munkres \(KM\)](#) 算法将聚类后的预测标签映射为真实标签，要求预测类别数 ≤ 真实类别数

Label = [0,0,0,1,1,1,2,2,2]
Pred = [2,2,0,1,1,0,1,0,0]
Pnew = [0,0,2,1,1,2,1,2,2]

[0,1] 区间内，
越大越好。

以下哪些是聚类分析的目标？

- ☐ A 得到较高的簇内相似度
- ☐ B 得到较低的簇间相似度
- ☐ C 使得簇间的距离尽可能小
- ☐ D 使得簇内样本与簇中心的距离尽可能小

提交

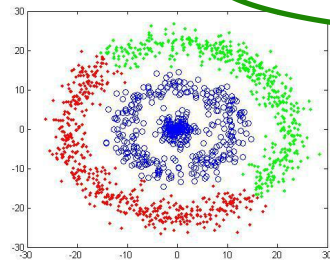
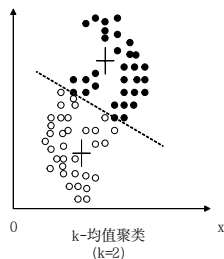
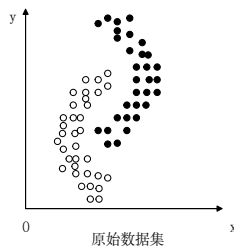
以下哪些是聚类分析的目标？

- ☒ A 得到较高的簇内相似度
- ☒ B 得到较低的簇间相似度
- ☐ C 使得簇间的距离尽可能小
- ☒ D 使得簇内样本与簇中心的距离尽可能小

提交

- k-均值算法不适用于非凸面形状（非球形）的数据集**，例如下图2个例子，**k-均值算法的聚类结果与真实数据类别有非常大的差别**

背景知识：如果 S 中任两点的连线内的点都在集合 S 内，那么集合 S 称为凸集。反之，为非凸集。



k-means 聚类 (k=3)

月牙形、圆环形、中空、有凹的数据集都不是凸集。

聚类问题

K-means

高斯混合聚类

密度聚类

层次聚类

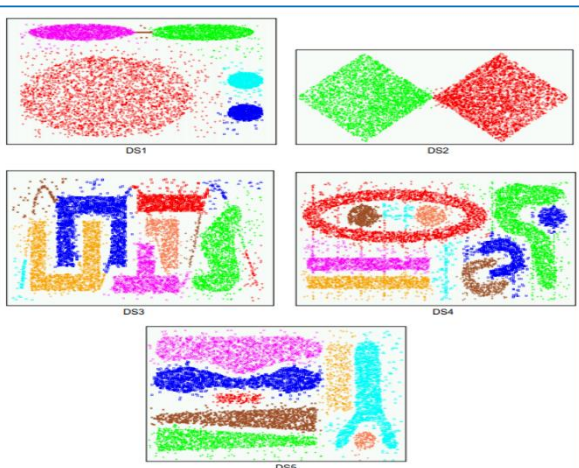


Figure 10: The clusters discovered by CHAMELEON for the five data sets.

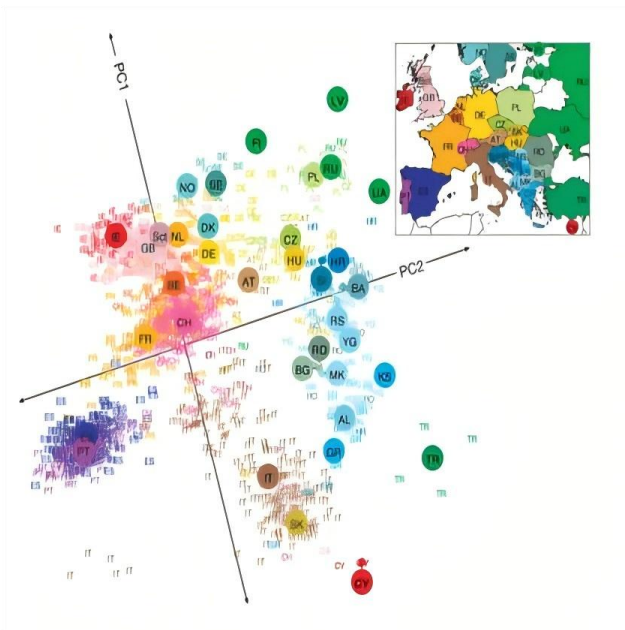
DB1~DB5中哪些是凸型数据集？

- A. DB1 B. DB2
C. DB3 D. DB4
E. DB5

✗ 不适合使用K-Means的场景

场景特征	问题分析	替代方案
非球形簇	K-Means无法识别环形、流形等复杂形状	DBSCAN、谱聚类
簇大小差异很大	会偏向大簇，小簇可能被忽略	层次聚类
噪声数据多	对异常值敏感，质心会被拉偏	DBSCAN、鲁棒聚类
类别型数据	欧氏距离不适用于类别变量	K-Modes、层次聚类
簇数量未知	需要反复试验确定K值	肘部法则、轮廓系数辅助

降维技术 - 化繁为简的艺术



当数据“太胖”了怎么办？

测量了人体内2万个基因的表达水平，找出哪些基因对某种疾病最重要。
有没有问题？

有问题：

- 1、处理速度慢
- 2、可视化困难

人类难以理解三维以上的空间

- 3、“维度诅咒”

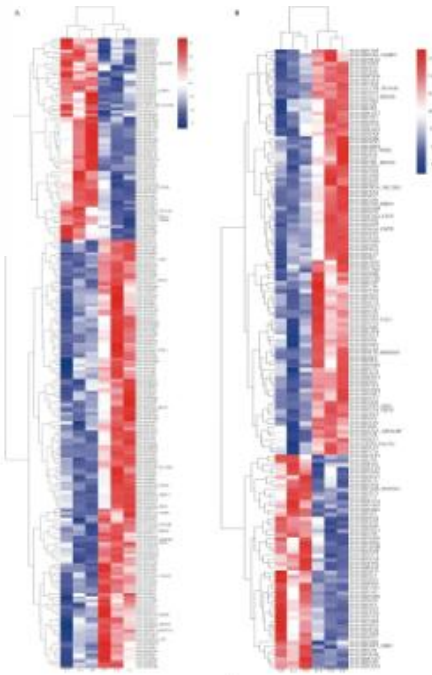
机器学习模型容易过拟合且效果差

- 4、特征冗余与噪音

很多特征相关性强或包含无用信息



DNA from set of
individuals



为什么我们需要降维?

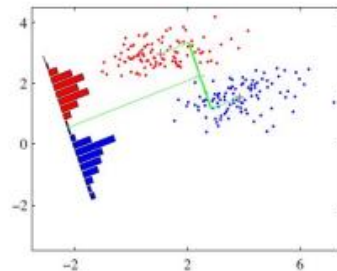
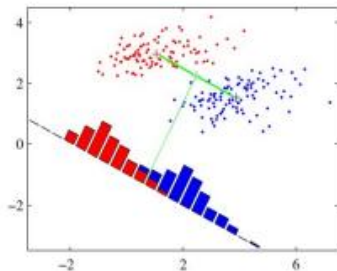
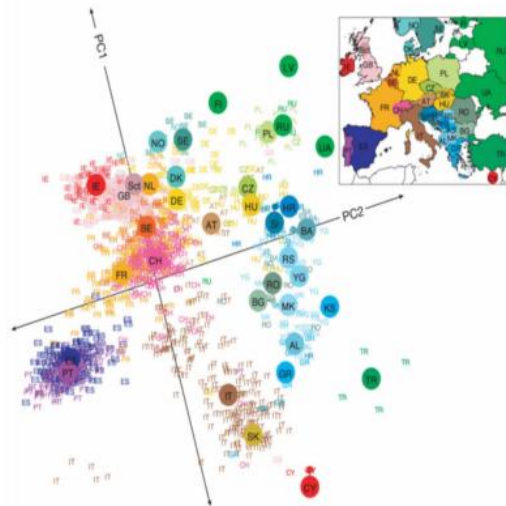
- 数据可视化 (二维绘图)
- 数据去噪, 去除系统性伪影 (生物学中很重要)
- 数据压缩 (图像, 音频)
- 加速监督学习

降维

- 高维数据转换为低维表示
- 保留原始数据的关键结构和重要信息

挑战

如何在保留数据本质的同时简化其表示?

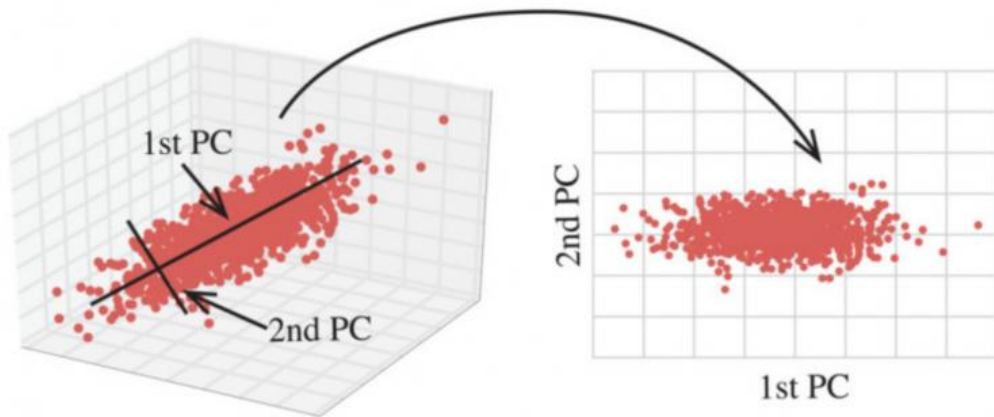


主成分分析 (Principal Component Analysis, PCA)

经典的线性降维方法，通过寻找数据中的主要变化方向来实现降维。

核心思想

将高维数据线性变换到新的坐标系统中，使其方差最大化，同时保持不同维度之间的正交性（寻找数据方差最大的正交方向序列）。



第一主成分是数据方差最大的方向，即协方差矩阵最大特征值对应的特征向量。
第二主成分是与第一主成分正交且方差次大的方向，依此类推。

主成分是原始维度的线性组合（即“新维度”），而非直接选取某些原始维度

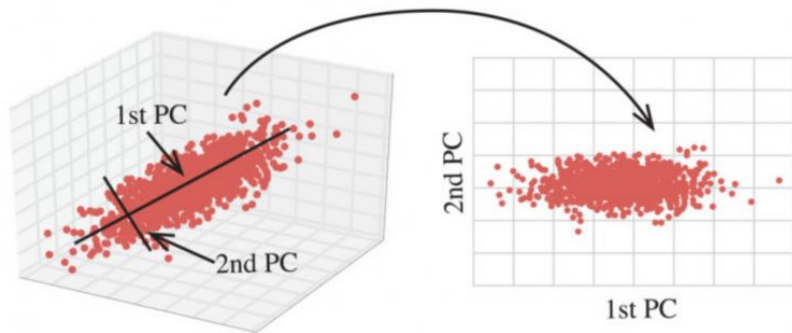
原始维度是数据表中直接观测的列，物理意义明确（如数学成绩、英语成绩）

主成分是将原始维度按权重重新组合，生成的新变量

如： $0.7 \times \text{数学} + 0.3 \times \text{英语}$

主成分是数学构造的“综合指标”，可以依据给每个主成分一个有意义的解释

- PCA 通过寻找投影后数据方差最大的方向（主成分）来保留最重要的信息。
 - 方差大的方向数据分布更分散，信息量更大；
 - 方差小的方向可能是噪声或冗余信息。
- PCA 常见选择标准：保留 85%~95% 的累积方差。例如：前 2 个主成分贡献了 90% 的方差，则降维到 2 维即可。



◆ 用主成分分析将这组二维数据降至一维: $X = \begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix}$

➤ 1) 因为这组数据已经是零均值了, 故中心化这步直接跳过

➤ 2) 求协方差矩阵 $\Sigma = \frac{1}{5} \begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1 & -2 \\ -1 & 0 \\ 0 & 0 \\ 2 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{6}{5} & \frac{4}{5} \\ \frac{4}{5} & \frac{6}{5} \end{pmatrix}$

➤ 3) 特征值分解: 求得特征值 2 和 $2/5$

求得特征向量为 $\begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}, \begin{pmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$

$$|A - \lambda E| = \begin{vmatrix} \frac{6}{5} - \lambda & \frac{4}{5} \\ \frac{4}{5} & \frac{6}{5} - \lambda \end{vmatrix} = \left(\frac{6}{5} - \lambda\right)^2 - \frac{16}{25} = (\lambda - 2)(\lambda - 2/5) = 0$$

➤ 4) 选取最大特征值组成投影矩阵

➤ 5) 数据降维/投影:

$$Y = (1/\sqrt{2} \quad 1/\sqrt{2}) \begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} -3/\sqrt{2} & -1/\sqrt{2} & 0 & 3/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$$

基本流程：

Step 1: 数据准备

Step 2: 数据标准化

Step 3: 应用PCA

Step 4: 分析主成分

Step 5: 降维后应用



山鸢尾



变色鸢尾



维吉尼亚鸢尾

花萼长度	花萼宽度	花瓣长度	花瓣宽度	品种 (标签)
5.1	3.3	1.7	0.5	0 (山鸢尾)
5.0	2.3	3.3	1.0	1 (变色鸢尾)
6.4	2.8	5.6	2.2	2 (维吉尼亚鸢尾)

#1 : 准备数据

```
import numpy as np
import pandas as pd
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
data = pd.read_csv('iris_data.csv')
data.head()
X = data.drop(['target', 'label'], axis = 1)
y = data.loc[:, 'label']
```

	sepal length	sepal width	petal length	petal width	target	label
0	5.1	3.5	1.4	0.2	Iris-setosa	0
1	4.9	3.0	1.4	0.2	Iris-setosa	0
2	4.7	3.2	1.3	0.2	Iris-setosa	0
3	4.6	3.1	1.5	0.2	Iris-setosa	0
4	5.0	3.6	1.4	0.2	Iris-setosa	0

2: #对原始数据X进行标准化 (均值为0, 方差为1)

```
from sklearn.preprocessing import StandardScaler
X_norm = StandardScaler().fit_transform(X)
```

3: #应用PCA

```
from sklearn.decomposition import PCA
pca = PCA(n_components = 4) # 也可以写成pca = PCA()
X_pca = pca.fit_transform(X_norm)
```

n_components = 4 将原数据降维到 4个主成分 (输出4个新特征)



山鸢尾



变色鸢尾



维吉尼亚鸢尾

4: #分析主成分

```
var_ratio = pca.explained_variance_ratio_  
print(var_ratio)
```

#获取各主成分的方差贡献率 (各成分重要性)

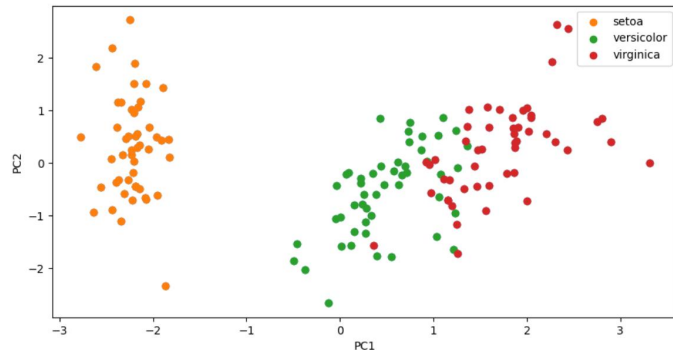
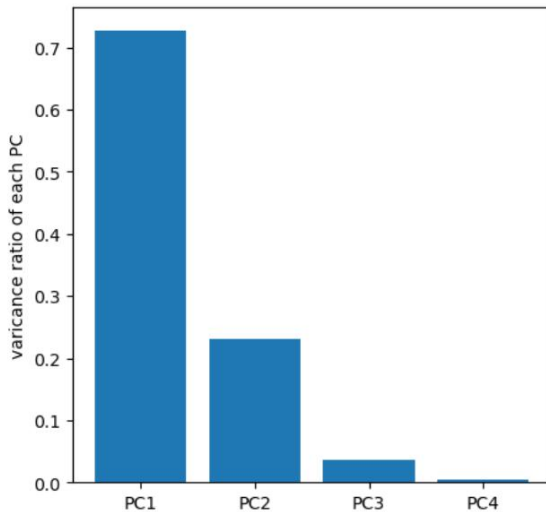
#常见选择标准: 保留85%~95%的累积方差

```
[0.72770452 0.23030523 0.03683832 0.00515193]
```

5: #基于降维后数据计算模型准确率

```
pca = PCA(n_components=2)  
X_pca = pca.fit_transform(X_norm)  
svm_classifier = SVC(kernel='rbf', random_state=42)  
svm_classifier.fit(X_pca, y)  
y_predict = svm_classifier.predict(X_pca)  
accuracy = accuracy_score(y, y_predict)  
#如果准确率下降不多, 说明PCA选择合理
```

正确率: 降维前 0.9733 -> 降维后 0.92



降维是“有损压缩”

会有信息损失，关键在于权衡：

用更少维度换取计算效率、可视化可能性和模型性能的提升

尽量保留最有价值的信息

没有“最好”的方法

选择取决于：

数据特点（线性、非线性？）

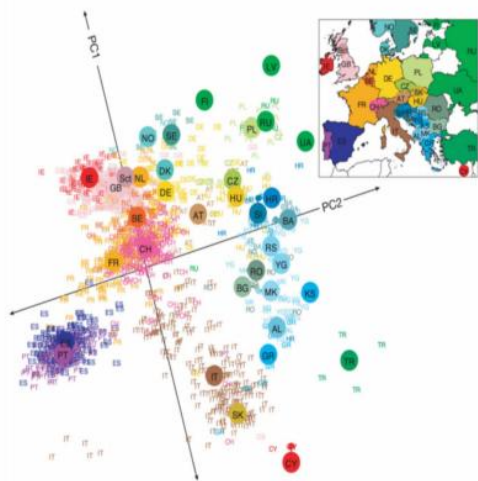
目标（可视化、压缩、下游任务？）

计算资源

降维是AI的基石之一

处理海量高维数据成为可能

多领域核心预处理和分析工具（机器学习、数据挖掘、模式识别）





- 1.请举例说明监督机器学习和无监督机器学习之间的区别。
- 2.传统的K-Means算法中，请说明初始的聚类中心以及第一次迭代后的聚类中心是怎么确定的？
- 3.请解释PCA降维的核心思想。
- 4.请说明KNN和K-Means的异同点。